

Dokumentacja Projektowa Projektu C

System wyznaczania współrzędnych węzłów grafów planarnych

Autorzy: Mateusz Bombola, Kajetan Karwacki

26 maja 2026

Spis treści

I Dokumentacja Funkcjonalna	4
1 Cel projektu	4
2 Opis algorytmów rozmieszczania grafu	4
3 Interfejs linii poleceń	4
3.1 Lista dostępnych flag	4
4 Formaty danych	5
4.1 Format wejściowy	5
4.2 Format wyjściowy	5
5 Kody błędów	5
6 Przykłady użycia	5
 II Dokumentacja Implementacyjna	 6
7 Architektura systemu i podział na moduły	6
7.1 Moduły języka C	6
7.2 Moduły testowe i narzędziowe	6
8 Struktury danych	6
9 Implementacja algorytmów optymalizacyjnych	7
9.1 SMACOF (smacof.c)	7
9.2 Fruchterman-Reingold (fruchterman.c)	7
10 Specyfikacja wejścia i wyjścia	7
10.1 Wejście	8
10.2 Format plików tekstowych (txt)	8
10.3 Format plików binarnych (bin)	8
11 Zarządzanie pamięcią i łączenie	8
 III Dokumentacja Projektowa	 9
12 Wstęp i podsumowanie realizacji	9
13 Metodologia testowania	9
14 Porównanie algorytmów i wyniki testów	9
14.1 Analiza wydajności i czasu wykonania	9
14.2 Ocena jakości wizualnej	9

Część I

Dokumentacja Funkcjonalna

1 Cel projektu

Celem projektu jest stworzenie aplikacji w języku C, która na podstawie topologii grafu planarnego wyznacza optymalne współrzędne dwuwymiarowe (x, y) dla jego wierzchołków. Program służy jako silnik obliczeniowy przetwarzający dane o połączeniach w celu wygenerowania układu umożliwiającego czytelną wizualizację grafu.

2 Opis algorytmów rozmieszczania grafu

W programie zaimplementowano dwie niezależne metody wyznaczania współrzędnych:

- **Algorytm SMACOF (smacof):** Służy do rozmieszczania elementów w przestrzeni na podstawie ich wzajemnych podobieństw lub odległości. Działa poprzez iteracyjne poprawianie położenia punktów tak, aby odległości między nimi jak najlepiej odpowiadały zadanym wartościom. W efekcie powstaje układ, w którym elementy bardziej podobne znajdują się bliżej siebie, a mniej podobne dalej.
- **Algorytm Fruchtermana-Reingolda (fr):** Automatycznie rozmieszcza elementy grafu w przestrzeni tak, aby był on czytelny i przejrzysty. Traktuje węzły jak obiekty, które wzajemnie się odpychają, a połączone elementy przyciągają się jak sprężyny. Dzięki wielokrotnemu przeliczaniu tych zależności program stopniowo ustala układ, w którym struktura grafu jest łatwa do zrozumienia.

3 Interfejs linii poleceń

Program jest sterowany poprzez argumenty przekazywane w wierszu poleceń. Składnia wywołania:

```
./graph_layout -i <input> -o <output> -a <smacof|fr> [-n iter] [-f txt|bin]
```

3.1 Lista dostępnych flag

Flaga	Argument	Opis
-i	<i>ścieżka</i>	Ścieżka do wejściowego pliku tekstowego (wymagane).
-o	<i>ścieżka</i>	Ścieżka do pliku wynikowego (wymagane).
-a	<i>algorytm</i>	Wybór algorytmu: smacof lub fr .
-n	<i>liczba</i>	Liczba iteracji (domyślnie 1000).
-f	<i>format</i>	Format wyjściowy: txt lub bin .
-h	—	Wyświetlenie pomocy technicznej.

Tabela 1: Parametry sterujące programem

4 Formaty danych

4.1 Format wejściowy

Każda linia reprezentuje krawędź: <nazwa> <od_id> <do_id> <waga>.

Przykład:

AB	1	2	1
BC	2	3	1
CD	3	4	1
DB	4	2	1.407

4.2 Format wyjściowy

Wynikiem działania programu ma być plik tekstowy lub binarny (w zależności od wyboru użytkownika) zawierający listę współrzędnych wierzchołków: <wierzchołek> <współrzędna_x> <współrzędna_y>.

Przykład:

1	0.0	0.0
2	1.0	0.0
3	1.0	1.0
4	0.0	1.0

5 Kody błędów

- **Kod 1:** Błąd otwarcia pliku wejściowego.
- **Kod 2:** Nieprawidłowy format danych wejściowych.
- **Kod 3:** Nieznany algorytm (oczekiwano smacof lub fr).
- **Kod 4:** Błąd alokacji pamięci.

6 Przykłady użycia

1. Wyznaczenie współrzędnych metodą Fruchtermana-Reingolda i zapis do tekstu:

```
./graph_layout -i dane.txt -o wynik.txt -a fr -n 1000 -f txt
```

2. Wyznaczenie współrzędnych metodą SMACOF i zapis binarny (500 iteracji):

```
./graph_layout -i siec.txt -o mapa.bin -a smacof -n 500 -f bin
```

Część II

Dokumentacja Implementacyjna

7 Architektura systemu i podział na moduły

System został zaprojektowany z zachowaniem zasad programowania modułowego. Kod źródłowy w języku C rozdzielono na osobne pliki, oddzielając logikę sterującą od implementacji poszczególnych algorytmów obliczeniowych. Dodatkowo zintegrowano środowisko testowe oparte na skryptach Python.

7.1 Moduły języka C

- **graph.h**: Główny plik nagłówkowy spinający projekt. Zawiera definicje struktur danych (**Vertex**, **Edge**), stałych matematycznych (np. **EPSILON**) oraz deklaracje prototypów funkcji obliczeniowych.
- **main.c**: Moduł inicjujący i sterujący. Parsuje argumenty linii poleceń, wczytuje i weryfikuje dane topologii, dynamicznie alokuje pamięć, inicjalizuje wstępne współrzędne, a po wykonaniu obliczeń zapisuje wyniki do plików.
- **smacof.c**: Niezależny moduł realizujący algorytm SMACOF. Główną logikę umieszczono w funkcji `calculate_smacof`, wspieranej przez matematyczną funkcję pomocniczą `safe_dist`.
- **fruchterman.c**: Moduł zawierający implementację siłowego algorytmu optymalizacyjnego wewnątrz funkcji `calculate_fruchterman`.

7.2 Moduły testowe i narzędziowe

Na potrzeby zapewnienia jakości aplikacji przygotowano zestaw narzędzi:

- **generator.py**: Generator pseudolosowych instancji grafów. Generuje siatki połączeń dla zadanej liczby węzłów i krawędzi (z opcją losowania wag), produkując pliki wejściowe w formacie akceptowalnym przez program.
- **wizualizacja.py**: Analizator graficzny wykorzystujący bibliotekę `matplotlib`. Wczytuje oryginalną siatkę krawędzi oraz wygenerowane pozycje X i Y, umożliwiając wizualną weryfikację węzłów na płaszczyźnie.

8 Struktury danych

Podstawowymi strukturami wykorzystywanymi do reprezentacji grafów są:

```
typedef struct {
    int id;
    double x, y;
} Vertex;

typedef struct {
```

```
int u, v;
double weight;
} Edge;
```

Mechanizm mapowania węzłów (main.c): Numery węzłów (identyfikatory) podane w pliku wejściowym nie zawsze występują po kolei (np. mogą to być węzły 10 i 100 bez żadnych węzłów pomiędzy). Moduł parsujący buduje strukturę `node_map`, przypisując takim węzłom kolejne, ciągłe indeksy $[0, N - 1]$. Pozwala to programowi optymalnie używać pamięci i zapobiega tworzeniu ogromnych, pustych tablic. Dopiero przy zapisie wyników do pliku, program przywraca węzłom ich oryginalne identyfikatory.

9 Implementacja algorytmów optymalizacyjnych

9.1 SMACOF (smacof.c)

Zastosowano algorytm oparty na transformacji Guttmana, składający się z kluczowych faz:

- **Macierz odległości:** Na bazie wczytanych krawędzi algorytm najpierw używa metody **Floyda-Warshalla** do propagacji najkrótszych ścieżek między wszystkimi nie-sąsiadującymi węzłami grafu, obsługując izolowane klastry dużą stałą odległością.
- **Inicjalizacja współrzędnych:** Zamiast losować pozycje startowe, program na samym początku układa wszystkie węzły równomiernie na obwodzie koła (korzystając z funkcji `sin()` i `cos()`). Zapobiega to nakładaniu się punktów na siebie i pomaga algorytmowi płynniej znaleźć optymalny układ, bez ryzyka utknięcia w złym ułożeniu początkowym.
- **Zabezpieczenie przed dzieleniem przez zero:** Jeśli dwa węzły znajdują się dokładnie w tym samym punkcie (odległość wyniesie 0), wzory algorytmu wygenerowałyby błąd matematyczny psujący wyniki. Aby temu zapobiec, wprowadzono funkcję `safe_dist`, która podmienia zerową odległość na minimalną dopuszczalną wartość (`EPSILON = 1e-9`), zapewniając stabilność całego programu.

9.2 Fruchterman-Reingold (fruchterman.c)

Zaimplementowano klasyczny model sprężyn i odpychających się ładunków elektrycznych. Obliczenia na każdą iterację `iterations` dzielą się na:

- Faza $O(V^2)$: globalne odpychanie poszczególnych wierzchołków na zasadzie odwrotnej proporcjonalności do ich odległości.
- Faza $O(E)$: iteracja wzdłuż bezpośrednich krawędzi (zdefiniowanych w wektorze struktur `Edge`) i przyciąganie połączonych wierzchołków wzajemnymi wektorami sił z jednoczesnym uwzględnieniem wagi (`weight`).

10 Specyfikacja wejścia i wyjścia

Poniższa specyfikacja definiuje formaty danych wymienianych między modułem obliczeniowym, a zewnętrznymi komponentami systemu.

10.1 Wejście

Program parsuje pliki wejściowe liniowo wywołując instrukcję `fscanf`. Kod zawarty w `main.c` aplikuje automatyczną naprawę błędnych danych. Dowolne wejściowe wagi krawędzi ≤ 0 są interpretowane i nadpisywane systemową stałą o wartości 0.1, co zapobiega propagacji błędu przez ujemne odległości.

10.2 Format plików tekstowych (txt)

Wynikowa lokalizacja wierzchołków generowana jest do postaci ASCII i powinna być parsowana wyrażeniem: `%d %f %f\n` (Kolejno: Integer ID, Double Oś X, Double Oś Y).

10.3 Format plików binarnych (bin)

Opcjonalny zrzut strukturalny oparty o procedurę `fwrite (save_to_binary)`. Wymaga zastosowania odpowiednich technik deserializacji:

1. **Liczba wierzchołków (N):** Pierwsze 4 bajty pliku to zwykła liczba całkowita (typ `int`). Informuje ona, ile łącznie węzłów zapisano w pliku.
2. **Dane pojedynczego wierzchołka:** Reszta pliku natomiast wypełniona jest blokami 20-bajtowymi. Architektura bloku to: 4 bajty przestrzeni id (Signed Integer 32-bit), 8 bajtów x (Double 64-bit) oraz 8 bajtów y (Double 64-bit).

11 Zarządzanie pamięcią i łączenie

Kluczowe wektory i macierze przetrzymywane są na stercie. Główna tablica struktur węzłowych `vertices` jest alokowana z użyciem `malloc` przez `main.c` (tylko na podstawie wyliczonej ilości realnie zidentyfikowanych wierzchołków, nie na stałym przedziale rozmiaru grafu). Ponadto tablice dwuwymiarowe w funkcji `calculate_smacof` (odległości i wagi) są budowane jako macierze wskaźników dynamicznych.

Wszystkie dynamicznie zaalokowane zasoby są zwalniane za pomocą wywołań funkcji `free()` po zakończeniu działania algorytmów lub tuż przed zamknięciem programu, co zapobiega wyciekowi pamięci.

Komenda kompilacji z wymuszeniem linkowania bibliotek współdzielonych dla trygonometrii:

```
gcc -o graph_layout main.c smacof.c fruchterman.c -lm
```


Część III

Dokumentacja Projektowa

12 Wstęp i podsumowanie realizacji

Zgodnie z wymaganiami projektowymi, program został w pełni zaimplementowany, przetestowany i jest gotowy do integracji z docelowym interfejsem graficznym w języku Java. Zrealizowano i przetestowano dwa algorytmy wyznaczania współrzędnych: model Fruchtermana-Reingolda oraz algorytm SMACOF.

13 Metodologia testowania

Aby porównać zaimplementowane algorytmy, wykorzystano dedykowane środowisko testowe oparte na skryptach w języku Python:

- **Skrypt generujący (generator.py):** Pozwolił na masowe tworzenie grafów o różnej gęstości, liczbie węzłów oraz losowych wagach krawędzi. Umożliwiło to zbadanie stabilności programu w języku C przy dużych obciążeniach.
- **Skrypt wizualizacyjny (wizualizacja.py):** Moduł oparty na bibliotece `matplotlib`, wizualizujący wygenerowane przez program C współrzędne na płaszczyźnie, co pozwoliło na ocenę jakości układu grafu.

14 Porównanie algorytmów i wyniki testów

14.1 Analiza wydajności i czasu wykonania

Zarówno SMACOF, jak i algorytm Fruchtermana-Reingolda wykazują odmienną złożoność obliczeniową. Poniższa tabela przedstawia uśrednione czasy wykonania dla różnych rozmiarów grafów (przy stałej liczbie 1000 iteracji):

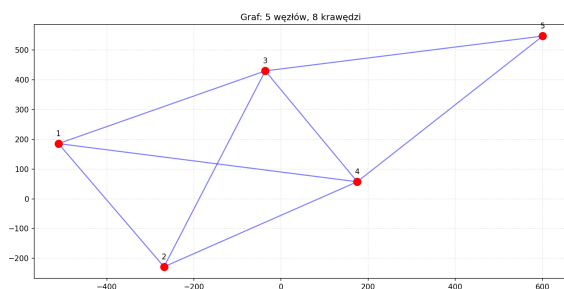
Algorytm	Mały graf (20 węzłów)	Średni graf (100 węzłów)	Duży graf (500 węzłów)
Fruchterman-Reingold	29 ms	428 ms	7201 ms
SMACOF	23 ms	308 ms	15284 ms

Tabela 2: Porównanie wydajności algorytmów (1000 iteracji)

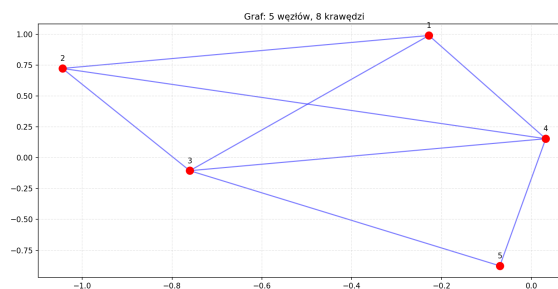
Wniosek wydajnościowy: Na podstawie tabeli nr. 2 widać, że algorytm Fruchtermana-Reingolda radzi sobie gorzej z mniejszymi grafami, jednak wyprzedza algorytm SMACOF przy bardzo dużej ilości węzłów.

14.2 Ocena jakości wizualnej

Poniższe grafiki przedstawiają ten sam graf wejściowy przetworzony przez oba zaimplementowane algorytmy.



Rysunek 1: Wynik alg. Fruchtermana-Reingolda



Rysunek 2: Wynik alg. SMACOF

Wizualna analiza wygenerowanych układów wykazuje, że:

- **Fruchterman-Reingold** lepiej radzi sobie z równomiernym rozkładaniem wierzchołków, tworząc symetryczne układy ułatwiające analizę topologii.
- **SMACOF** wykazuje wyższą precyzję w odzwierciedlaniu różnic w wagach krawędzi (zachowanie proporcji metrycznych)..

15 Napotkane problemy i zastosowane rozwiązania

Podczas implementacji zidentyfikowano i rozwiązano następujące wyzwania techniczne:

1. **Niestabilność numeryczna przy nakładających się wierzchołkach:** W obu modelach nakładające się na siebie węzły (odległość równa 0) powodowały dzielenie przez zero. Rozwiązaniem było zaimplementowanie funkcji `safe_dist`, podmieniającej zerową odległość na minimalną stałą `EPSILON`.
2. **Optymalizacja zużycia pamięci:** Pliki testowe posiadały rzadkie, nieciągłe identyfikatory (np. węzeł 10 i 1000). Bezpośrednia alokacja doprowadziłaby do ogromnego marnotrawstwa pamięci. Zastosowano mechanizm wektorowego mapowania (`node_map`), który spłaszczył indeksy na ciągły obszar roboczy.
3. **Problem minimów lokalnych:** Losowa inicjalizacja współrzędnych często powodowała zatrzymanie się algorytmów w nieestetycznym, zaplątanym układzie. Zmiana geometrii startowej na równomierne rozmieszczenie wierzchołków na okręgu znacząco zwiększyła odsetek udanych optymalizacji.